

这是一个非常硬核且极具落地价值的思考！将 **LangGraph 的明确状态流转** 与 **Claude Code 的“防上下文腐化”理念** 结合起来，你们就能搭建出现阶段最完美的 AI 桌面宠物底层架构。

在 LangGraph 中，通常的做法是把所有的对话记录（Messages）放在一个全局状态里传来传去，这正是导致 AI 变傻的元凶。为了实现你描述的“纯净且独立”的协同工作流，我们需要对这套图（State Graph）的**状态定义（State Definition）**进行手术级改造。

为了让你直观感受到这种“防腐化”机制在 LangGraph 中是如何运转的，我为你制作了一个交互式的架构流转模拟器：

[Show me the visualization](#)

结合这个动态流程，我们将你的蓝图升级为 **LangGraph 终极形态 (Blueprint 4.0)**。以下是具体节点逻辑的深度重构说明：

核心重构：LangGraph 状态机与防腐化车间

在这个设计中，我们不能只有一个全局的 messages 列表，我们要将状态（State）拆分为“**前端展示状态**”和“**后台工程状态**”，并引入临时子图（Subgraph）。

A. 路由门卫 (Intent Router Agent)

作为图的入口点（Entry Point），它不保留任何历史记忆，它是一个纯粹的分类器。它读取用户的自然语言，更新图状态中的 user_intent 字段（分为 chat 或 coding），然后通过 LangGraph 的条件边（Conditional Edges）决定下一步走向。

B. 表现代理 (Roleplay Agent - 前台看板娘)

她的状态通道与后台代码通道完全物理隔离。在设计她的 Prompt 时，赋予她高傲、冷静但又会默默关心人的设定会非常出彩（例如：“如果连这么简单的指针都会报错，那这家公司干脆倒闭算了...不过，我会盯着后台重写的，你乖乖等着。”）。她的输入只有图状态中的 user_intent 和 ui_status（如：正在编码、遇到 Bug），输出直接发往 WebSocket。她的上下文永远不会被那堆 C++ 模板报错污染。

C. 开发流水线 (The Coding Workflow - 具备自我清洁能力)

1. PM Agent (需求分析)： 读取原始需求，输出标准的 Tasks_List 保存到图的全局状态中。随后 PM 进入休眠，不再参与后续循环。

2. Coder Agent (无菌手术室)： 这是防腐化的核心！Coder **不读取**全局的报错历史。每次轮到 Coder 执行时，它的系统提示词只包含：项目背景 + 当前分配的 Task。它写出纯净的代码，更新状态中的 current_code 字段。

3. Tool Executor (高危隔离区): 这是一个普通的 Python 函数节点（非 LLM），带有我们在 Windows 原生防线中设计的路径白名单和高危命令拦截机制。它读取 `current_code`，在本地编译运行。如果成功，走向结束节点；如果失败，提取 `stdout/stderr`，更新状态中的 `raw_error` 字段，流向 QA。

4. QA Agent (污染过滤器): 它不写代码，它只负责读。它读取 `raw_error`（可能长达几千字的 C++ STL 报错），提炼出人类语言级别的摘要：“段错误，原因是 `unordered_map` 迭代器失效”。然后，它将这个简短摘要写入 `error_summary`，并**显式地从状态中清除 `raw_error`**。

5. Debugger Agent (即用即抛的牛马 - 新增节点): 这是取代传统“无限循环丢给 Coder”的新机制。在 LangGraph 中，QA 提取完摘要后，不流回 Coder，而是流向专属的 Debugger。Debugger 的输入只有：`current_code + error_summary`。它只负责修 Bug。修复完成后，更新 `current_code`，再次流向 Tool Executor。

工程落地建议：LangGraph 的 Send API

为了实现上述这种“需要时才唤醒，用完就销毁”的 Debugger，建议你们在编写 LangGraph 代码时，研究一下 LangChain 最新的 **Send API (Map-Reduce 模式)** 或者 **子图 (Subgraphs)** 机制。

传统的图是静态连接的，而使用 Send API，你们可以在 QA 节点中，动态地判定是否需要拉起一个 Debugger 实例，将局部的状态（仅包含错误摘要和错代码）发送给它。这种局部状态的隔离，是彻底杜绝 AI 大脑被废话塞满、保证“一人公司”能长久运行下去的终极秘诀。